Making attention cheap

FlashAttention, MQA, GQA, MLA, sliding windows — and how to choose

Here is a bill that sneaks up on everyone. Your 7B model's weights fit comfortably in 13.5 GB. Then the product team asks for 32K context, serving wants batches of 32 concurrent users, and you do the arithmetic from Chapter 3.7: 512 KB of KV cache per token, times 32,768 tokens, times 32 sequences — **550 GB**. For the *cache*. Forty times the model itself, spread across a rack of GPUs that are otherwise idle, because decoding is not compute-bound at all: every generated token has to *read* that entire cache from memory, and the arithmetic per byte read is trivial. Meanwhile, on the training side, the same mechanism is running up a different bill: attention is the only component of the transformer whose cost curve bends with sequence length, and past a crossover you can compute exactly (we will), it grows quadratically while everything else stands still.

This chapter is about the five-year engineering campaign against those two bills, and it is the part of the architecture where the frontier labs have diverged most visibly: Llama and Qwen went one way (grouped queries), DeepSeek went another (latent compression), Mistral and Gemma a third (sliding windows), and everyone adopted FlashAttention underneath. Each variant is a different answer to the same question — *which parts of full attention are you willing to give up, and for what?* — and by the end you will be able to read any model's attention block from its config and price it, and to choose one for your own model with a clear head. Every variant here also has a live animation in the attention hub: same tokens, same decoding loop, watch the cache shrink as you switch mechanisms.

13.1 Why attention is the bottleneck

Start by locating the enemy precisely, because "attention is quadratic" is true but is not the whole story, and the fixes in this chapter each target a different part of it. Attention generates three distinct costs, and they bind in different regimes.

The quadratic FLOPs. From Chapter 3.2: computing scores $QK^\top$ and applying them to V costs about $4L^2d$ FLOPs per layer for the whole sequence — per *token*, that is $4Ld$, growing linearly with how much context each token looks at. Everything else in the layer — the four attention projections and the SwiGLU MLP — costs a fixed $\approx 24d^2$ per token regardless of L (for Llama-2-7B geometry: 405 MFLOPs). Set the two equal and the crossover falls out:

$$4Ld = 24d^2 \quad \Rightarrow \quad L^* \approx 6d.$$

For $d = 4096$ that is $L^* \approx 25{,}000$ tokens. Below it, the transformer is a matmul machine and attention's pairwise term is a rounding error; above it, attention progressively *becomes* the model's cost. That single number explains a lot of behavior you will observe in the wild: why 4K-context pre-training barely notices attention, why the long-context

extension phase (Chapter 21) suddenly does, and why the sliding-window and sparse designs of Section 13.4 come from exactly the labs that pushed context hardest.

The training memory. A naive implementation materializes the $L \times L$ score matrix per head, in the forward pass and again for the backward. At $L = 32\text{K}$ with 32 heads that is a terabyte-scale intermediate per batch element — which is why, before 2022, "long context" mostly meant "out of memory." This cost is an artifact of *how* attention was computed, not of the math, and Section 13.2 kills it without changing the model at all.

The decode bandwidth. The subtlest and, for serving economics, the dominant one.[1] When the model generates, each new token performs one query against the cached keys and values of every previous token (Chapter 3.7). The FLOPs are trivial — one row of the score matrix — but the *bytes* are not: the whole KV cache streams from HBM through the compute units for every single token produced. Generating 100 tokens at 32K context on our MHA 7B reads 100 × 16.8 GB ≈ 1.7 TB from memory to do almost no math. The operation's arithmetic intensity (Chapter 8) is near 1 FLOP per byte on hardware that wants hundreds, so the GPU idles at a few percent utilization, and throughput is set by memory bandwidth and cache size, not by TFLOP/s. Shrink the cache and decode gets faster in direct proportion — which is the entire motivation for Sections 13.3 and 13.4.

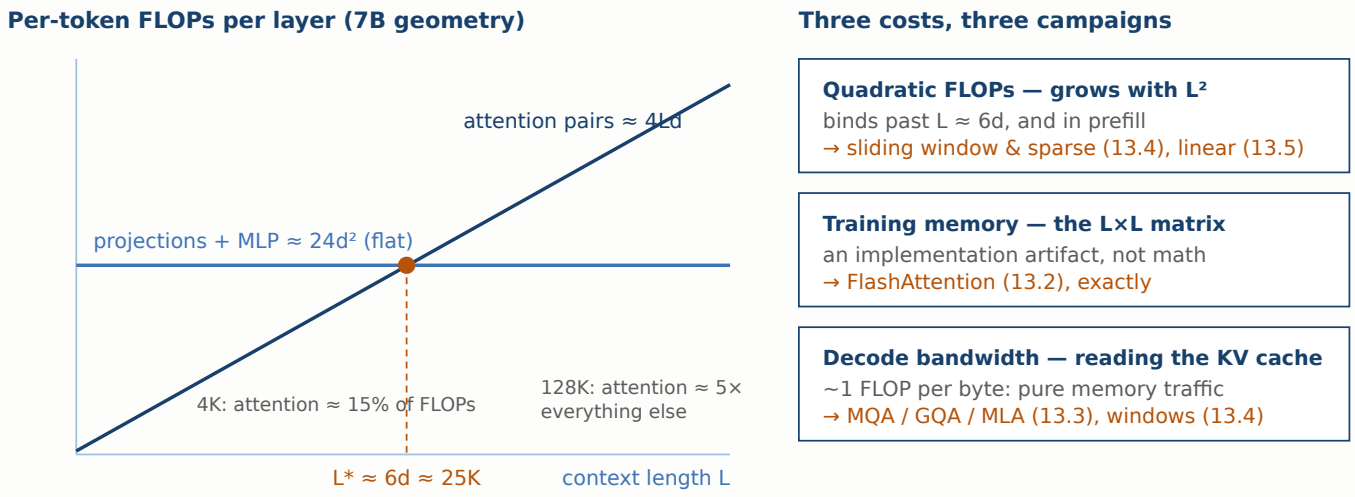

Figure 13.1 — Know which bill you are paying. Left: attention's per-token cost grows with context while the rest of the layer is flat; they cross near $L^* \approx 6d$ (~25K tokens at 7B geometry). Right: the three distinct costs and the section of this chapter that attacks each. A common mistake is deploying a fix for one bill while a different one binds.

One orientation note before the tour: everything in Sections 13.2 and 13.3 computes *exact* full attention or a re-parameterization of it — no token loses the ability to see any other token. Section 13.4 starts actually restricting who sees whom, and Section 13.5 replaces the mechanism outright. The further down the chapter you go, the more you save and the more you must verify you did not break.

13.2 FlashAttention: the tiling idea, properly

FlashAttention is the one item in this chapter that requires no design decision, changes no math, and belongs in every run: it computes bit-for-bit standard attention, just without ever writing the $L \times L$ matrix to memory.[2] To see how, you

need one fact about a GPU that Chapter 8 develops fully: it has a small, extremely fast on-chip memory (SRAM, ~200 KB per streaming multiprocessor) and a large, comparatively slow main memory (HBM, tens of GB). Naive attention makes three round trips through slow memory — write all scores, read them back for the softmax, read them again to multiply by V — and at long L the kernel spends its life waiting on those transfers, not computing.

The fix is tiling plus a classic streaming trick. Take a block of query rows and hold it in SRAM. Stream the keys and values through in blocks: for each K/V block, compute that tile of scores *on-chip*, fold it into a running result, and throw the tile away. The subtlety is the softmax, which normalizes over the whole row — seemingly impossible if you never hold the whole row. The *online softmax* solves it with two scalars per row: keep the running maximum m and the running sum of exponentials ℓ, and whenever a new block raises the max, rescale the accumulated output by $e^{m_{\text{old}} - m_{\text{new}}}$. Every block is processed once, the final output is mathematically identical, and the memory footprint drops from $O(L^2)$ to $O(L)$. For the backward pass, the score tiles are simply *recomputed* from Q and K rather than stored — the same recompute-over-store trade Chapter 15 applies to whole layers, here applied surgically inside one op.

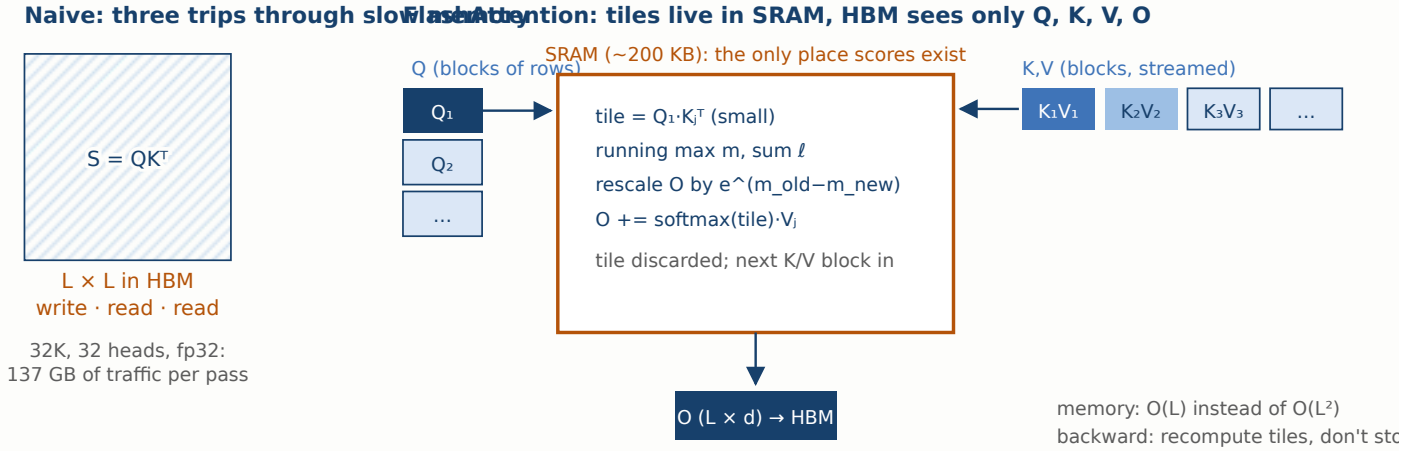

Figure 13.2 — The tiling idea. A block of queries is pinned in fast on-chip memory while key/value blocks stream past it; each score tile is computed, folded into the running output with the online-softmax rescaling, and discarded. The full score matrix never exists anywhere. The result is exact — this is an *IO optimization*, not an approximation. Watch it move: the attention hub's **FlashAttention** tab animates these tiles streaming between HBM and SRAM, with a live counter of the memory round trips a naive kernel would be making.

What it buys and what it doesn't. The original kernel reported 3× end-to-end speedup on GPT-2-scale training and a 15% improvement over the then-standing MLerf BERT record; FlashAttention-2 reorganized the work across thread blocks and roughly doubled throughput again, reaching ~70% of peak FLOPs on A100s.[23] Because it is exact, there is no quality decision to make and no evaluation to run — which is precisely why it is the only technique in this chapter that is simply *on* everywhere, exposed in PyTorch as `F.scaled_dot_product_attention` and in the varlen form whose `cu_seqlens` interface Chapter 16 uses for packing. Its limits matter too: it does not reduce the $4L^2d$ FLOPs (the pairs are all still computed — prefill at 128K is still expensive), it does nothing for the KV cache (decode is bandwidth-bound on cache *size*, and Flash doesn't shrink it), and it is a hand-tuned kernel, so exotic head dimensions or masks can silently drop you onto a slow path — one of Section 13.7's pitfalls. FlashAttention removes the artifact cost; the next two sections go after the real ones.

13.3 Shrinking the KV cache: MQA, GQA, MLA

Now the decode bill. The cache formula from Chapter 3.7 — $2 \cdot N \cdot h_{kv} \cdot d_{\text{head}} \cdot \text{bytes}$ per token — has exactly one knob that architecture can turn without touching the model's width or depth: h_{kv}, the number of key/value heads. The three designs in this section turn it progressively harder, and they share one crucial observation: the *queries* are where attention's expressiveness lives (each head asking a different question), while the keys and values are more like a shared library the questions are asked *of*. You can keep many questioners and consolidate the library.

MQA — one library for everyone

Multi-query attention is the blunt version: keep all h query heads, share a *single* key head and a single value head among them.[4]

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K^\top}{\sqrt{d_h}}\right) V, \qquad Q_i = XW_Q^{(i)}, \quad K = XW_K, \quad V = XW_V,$$

where $i = 1 \ldots h$ indexes query heads but there is only one W_K, W_V pair of shape $d \times d_h$. Symbols to keep straight from here on: h query heads, h_{kv} key/value heads (here 1), $d_h = d/h$ the head dimension. The cache shrinks by the full factor h — for a 32-head 7B, 512 KB/token becomes 16 KB/token — and the K/V projection parameters shrink too. PaLM, Falcon, and StarCoder shipped it.[5] The cost is real, though: all heads now attend over the same keys, so heads can differ only in *what they ask*, not in *how the sequence is indexed* for them, and measured quality drops noticeably on tasks that lean on retrieval; training can also be less stable. MQA is the right call when memory is desperate and the model is small; the field mostly moved one step back from the edge.

GQA — a library per group

Grouped-query attention is the interpolation that won. Partition the h query heads into $g = h_{kv}$ groups; each group shares one K/V head:[6]

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K_{\lceil ig/h \rceil}^\top}{\sqrt{d_h}}\right) V_{\lceil ig/h \rceil}.$$

Set $g = h$ and you have MHA; $g = 1$ and you have MQA; the sweet spot in practice is $g = 8$ almost regardless of h: Llama 3 uses 32 query heads over 8 KV heads at 8B and 64 over 8 at 70B; Mistral 7B is 32:8; Qwen 2.5 7B is 28:4.[7][8] The Ainslie paper's two findings made it the default: quality is statistically indistinguishable from MHA at 4–8× cache reduction, and — the operationally lovely part — you can *convert* an existing MHA checkpoint by mean-pooling its K/V heads group-wise and "uptraining" for about 5% of the original compute.[6] Trade-offs: the cache shrinks by h/g rather than h; the group count interacts with tensor parallelism (Section 13.7); and below $g = 4$ the MQA quality cliff starts to reappear. If you are designing a model today and have no exotic constraint, GQA-8 is the boring correct answer.

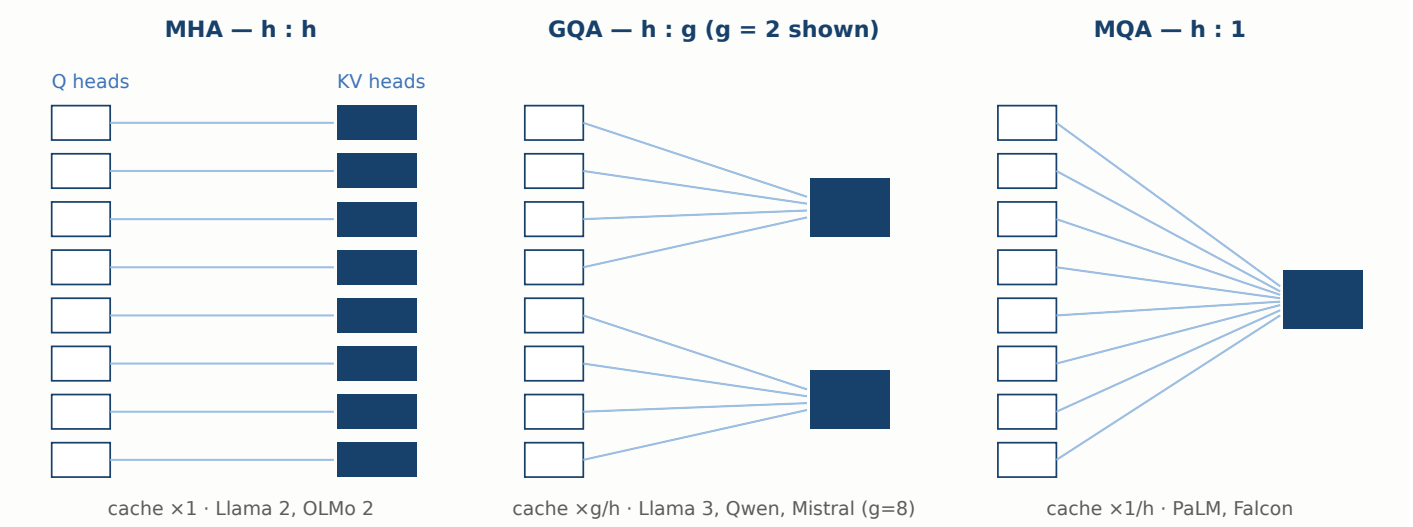

Figure 13.3 — Who shares whose library. White boxes are query heads; navy boxes are the KV heads whose keys and values get cached. Only the navy boxes cost cache memory at inference, so the design question is how many questioners can share an index before retrieval quality degrades. The empirical answer: about four to eight per group. Watch this as an animation in the attention hub.

MLA — cache the summary, not the library

DeepSeek's multi-head latent attention attacks the same bytes from a different angle: instead of sharing K/V heads, *compress* each token's key-value information into one small latent vector and cache only that.[9] Per token, a down-projection produces a latent $c^{KV} = W^{DKV}x$ of dimension d_c (512 in DeepSeek-V2/V3 — versus $2hd_h = 32{,}768$ for full MHA at their geometry); at attention time, per-head up-projections reconstruct keys and values:

$$c^{KV} = W^{DKV}x, \qquad k_i = W_i^{UK}c^{KV}, \qquad v_i = W_i^{UV}c^{KV},$$

with a separate small *decoupled* key of dimension $d_r = 64$ carrying RoPE (more on that in a moment). The apparent flaw — don't you have to decompress the whole cache every step, paying back everything you saved? — is dissolved by an algebraic trick worth admiring: since $q_i^\top k_i = (W_i^{UK\top} q_i)^\top c^{KV}$, the up-projection can be *absorbed into the query* once per new token, and attention runs directly against the latent cache. Nothing is ever decompressed. The cache is $d_c + d_r = 576$ elements per token per layer — a 57× reduction versus MHA at DeepSeek's 128-head geometry, about 70 KB per token across all 61 layers of the 671B DeepSeek-V3, which is *less than a 7B MHA model's 512 KB*.[9][10] And because the per-head keys are learned re-projections of a shared latent rather than pooled copies, DeepSeek reports quality *above* MHA, not below.

The costs are honest, too. MLA is the most complex mechanism in this chapter: extra projection matrices, the query-absorption identity to implement correctly, and — the famous wrinkle — RoPE breaks the absorption trick, because the position rotation sits between q and k and does not commute with the low-rank factors. Hence the decoupled design: a small rotary key (those 64 extra dimensions) carries position, the latent carries content, and the two score terms are summed. Off-the-shelf kernels and serving stacks needed real work to support all this, which is exactly the engineering price a 57× cache reduction buys.

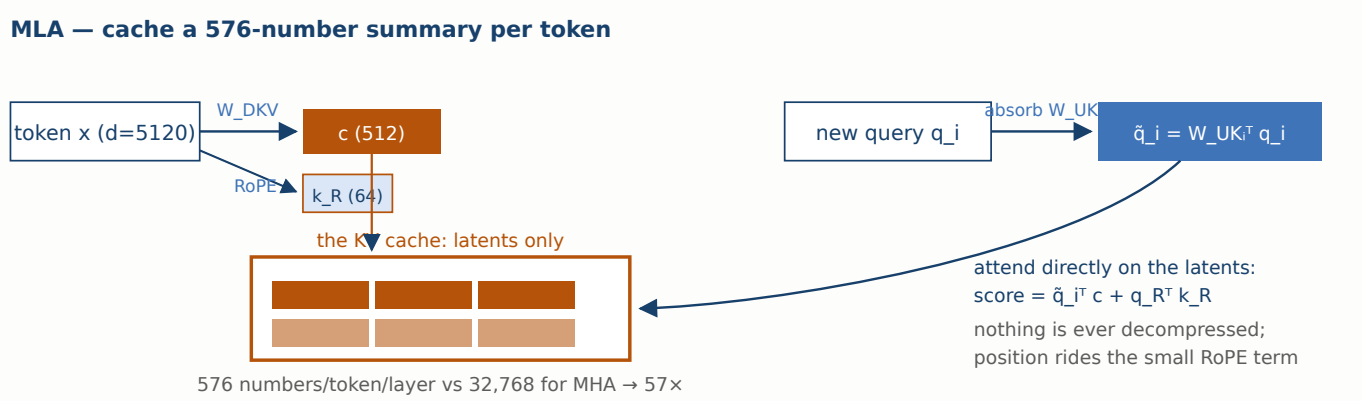

Figure 13.4 — Multi-head latent attention. Each arriving token is compressed to a 512-dim latent (plus a 64-dim decoupled RoPE key), and *that* is what the cache stores. The per-head key up-projection is folded into the query, so decoding attends straight into the latent cache. DeepSeek-V3's whole 671B cache costs ~70 KB per token — less than a vanilla 7B.[10]

13.4 Sliding windows and sparse attention

Everything so far preserved full attention: every token could still see every earlier token. The next lever gives that up, on a bet about language: most of what a token needs is nearby. *Sliding-window attention* (SWA) makes the bet literal — token t attends only to positions $t - W \ldots t$:[11]

$$\text{mask}_{tj} = \begin{cases} 0 & t - W \le j \le t \\ -\infty & \text{otherwise,} \end{cases}$$

turning the causal triangle into a diagonal band. Both bills fall at once: pairwise FLOPs drop from $O(L^2)$ to $O(LW)$, and the KV cache stops growing at W entries — a rolling buffer where the oldest token's cache line is simply overwritten. Mistral 7B shipped this with $W = 4096$.[8] The subtle saving grace is depth: information can hop window by window across layers, so the theoretical receptive field after N layers is $N \times W$ — 32 layers × 4K ≈ 131K positions. But "reachable in principle via multi-hop" is much weaker than "directly attendable," and pure SWA measurably degrades exact long-range retrieval — the needle-in-a-haystack failure mode. The design that survived contact with evaluation is the *hybrid stack*: Gemma 2 alternates local (W = 4096) and global layers 1:1, and Gemma 3 pushed to five local layers (W = 1024) per global one, cutting cache dramatically while the periodic global layers keep true long-range access alive.[12][13]

Beyond fixed bands lies learned sparsity: let the model decide which distant blocks matter. Strided and block patterns date to 2019,[14] and the modern revival — DeepSeek's natively-trainable sparse attention (NSA), trained sparse from scratch rather than sparsified after the fact, with hardware-aligned block sizes[15] — is where frontier long-context work is heading. We treat it as one paragraph rather than a section because the recipes are young and moving; the durable lesson is the pattern of the whole chapter: fixed structure (windows) is cheap and predictable, learned structure (sparsity) buys quality back at the price of kernels and training complexity.

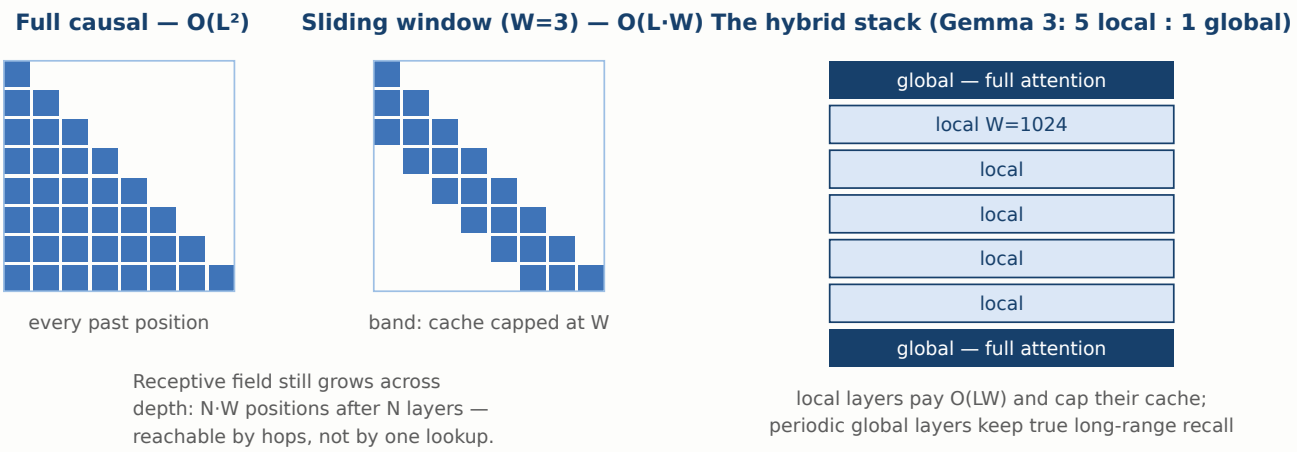

Figure 13.5 — Restricting who sees whom. Left: the causal triangle becomes a band; compute and cache both stop growing with context. Right: production systems rarely go all-local — they interleave windowed layers with full-attention layers, tuning the ratio (Gemma 2 used 1:1 at W=4096; Gemma 3, 5:1 at W=1024) to keep retrieval alive while most layers run cheap.[12][13]

13.5 Linear attention and hybrid architectures

The last rung abandons the softmax-over-pairs formulation entirely. If the attention kernel is factorizable — replace $\mathrm{softmax}(qk^\top)$ with $\phi(q)\phi(k)^\top$ for a feature map ϕ — then the sum over history collapses into a running matrix-valued *state* that is updated per token in $O(1)$: attention becomes a recurrence, cost linear in L, cache constant.[16] State-space models like Mamba arrive at a similar place from control theory, with input-dependent state dynamics that made the recurrent family competitive on language for the first time.[17] The catch has stayed stubbornly consistent: a fixed-size state is a lossy summary, and tasks that require exact recall of something specific said long ago — retrieval, long in-context learning, copying — are precisely where these models lag softmax attention, which can always look the fact up verbatim.

So the production form, again, is the hybrid: mostly linear/SSM layers for the cheap bulk, a few full-attention layers as the model's random-access memory — Jamba interleaved one attention layer per seven Mamba layers at 52B scale and reported near-parity with attention-only peers at a fraction of the cache.[18] This family is where the cost curve of the field is most likely to bend next, and it is moving fast enough that we confine it to this survey; the decision framework below treats "consider a hybrid" as the option you evaluate when even a windowed, GQA'd cache is still your binding constraint. Everything else in this book — the training loop, the data, the parallelism — applies to these architectures nearly unchanged, which is itself worth noticing: the transformer's *training recipe* outlived even proposals to replace its central mechanism.

13.6 What the big labs do

Table 13.1 is this chapter compressed to one screen, and it rewards reading as history: down the rows you watch the field walk away from vanilla MHA one step at a time, and the "cache per token" column is the reason why. Cache

figures are per token *per layer*, in BF16, computed from each model's published config with the formulas above — you can reproduce every number in the parameter counter.

Table 13.1 — Attention at the frontier. $h : h_{kv}$ is query heads : KV heads. Cache is per token per layer in BF16 (multiply by layers for the per-token total; e.g., Llama 2 7B: 16 KB × 32 = 512 KB). SWA entries also cap how many tokens are cached at all.

MODEL	MECHANISM	H : H_KV	WINDOW	CACHE / TOK / LAYER	WHY THEY CHOSE IT
Transformer (2017)[19]	MHA	8 : 8	full	2 KB	The original; nobody was decoding at scale yet.
Llama 2 7B (2023)[20]	MHA	32 : 32	full	16 KB	Kept MHA below 34B; GQA only at 34B/70B.
OLMo 2 7B (2024)[21]	MHA	32 : 32	full	16 KB	Research openness over serving cost at small scale.
PaLM 540B (2022)[5]	MQA	48 : 1	full	1 KB	Decode throughput at extreme scale; accepted the quality tax.
Mistral 7B (2023)[8]	GQA + SWA	32 : 8	4,096	4 KB, capped	Both levers at once: grouped heads and a rolling window.
Llama 3 8B / 70B (2024)[7]	GQA	32 : 8 / 64 : 8	full	4 KB / 4 KB	The industry default: ~MHA quality, 4–8× cache.
Qwen 2.5 7B (2024)[22]	GQA	28 : 4	full	2 KB	Pushed grouping harder (7 queries per KV head).
Gemma 2 9B (2024)[12]	GQA, local/global 1:1	16 : 8	4,096 (half of layers)	4 KB, half capped	Hybrid stack: cheap local layers, periodic global recall.
Gemma 3 (2025)[13]	GQA, local/global 5:1	e.g. 16 : 8	1,024 (5 of 6 layers)	mostly capped	Long context with a phone-sized cache.
DeepSeek-V3 671B (2024)[10]	MLA	128 q; latent	full	1.1 KB	57× vs its own MHA; quality ≥ MHA; max

MODEL	MECHANISM	H : H_KV	WINDOW	CACHE / TOK / LAYER	WHY THEY CHOSE IT
		512+64			engineering.
Jamba 52B (2024)[18]	Hybrid: Mamba + MHA 7:1	attn layers only	full (attn layers)	~⅛ of attn-only	Constant-state layers for bulk, attention for recall.

Three patterns to take away. First, GQA with about 8 KV heads is the modern center of gravity — Llama, Mistral, Qwen, and Gemma all sit there, differing only in ratio. Second, the two labs that deviated hardest, DeepSeek (MLA) and Google (aggressive local/global), are exactly the ones optimizing for cheap long-context serving as a product strategy — architecture follows the bill you care about. Third, full MHA has become a statement: OLMo keeps it for research comparability, and nobody keeps it for economics. On LatamGPT we followed the center of gravity — GQA-8 — precisely because Section 13.7's operational arguments (kernel maturity, tensor-parallel divisibility, uptraining escape hatches) all favor the crowded path unless your serving math screams otherwise.

13.7 Practical considerations

Attention variants are where architecture meets infrastructure, and most of the pain lives at that seam.

You can migrate to GQA; you cannot easily migrate away from your kernels. The Ainslie recipe — mean-pool the K/V heads of an MHA checkpoint into groups, then uptrain on ~5% of the original tokens — works well and is the standard escape hatch for a model trained before the decision was made.[6] Budget real evaluation for it: perplexity recovers quickly, but retrieval-heavy tasks recover last, and they are exactly what the cache reduction is stressing. Going the other direction (MLA, or adding windows) is a redesign, not a conversion — which is why the decision framework below insists these choices are made at design time.

Parallelism divisibility. Under tensor parallelism (Chapter 14), heads are split across GPUs, and h_{kv} must divide the TP degree or the KV heads get replicated — silently refunding part of the cache saving. GQA-8 across TP=8 puts exactly one KV head per GPU, one reason 8 is the magic number. Qwen's 28:4 at TP=8 replicates each KV head twice; fine, but know you are paying it.

The rolling buffer is a correctness trap. SWA's capped cache means position t overwrites $t - W$. Get the ring arithmetic wrong and the model attends to a stale token's K/V under a fresh position — no crash, subtly wrong generations. Test the cache implementation with a probe prompt longer than W whose correct answer requires the newest in-window token.

Fast paths are fragile. FlashAttention dispatches to tuned kernels keyed on head dimension (128 is the fast lane), mask type, and dtype. A custom additive mask, head_dim 96, or an fp32 fallback can silently cost you 3× — profile the attention op itself, not just end-to-end, after any change. And with packing (Chapter 16), use the varlen entry points with `cu_seqlens` rather than materializing block-diagonal masks, or you rebuild the memory problem Flash exists to remove.

Evaluate the thing you risked. Perplexity is nearly blind to cache-shrinking damage; what suffers is precise long-range recall. Any change in this chapter should gate on needle-in-a-haystack-style probes and long-document QA at full deployment context, decode throughput measured separately from prefill (they now scale differently), and — for windows and hybrids — a check at lengths *past* what any single window covers.

Common pitfalls

Fixing the wrong bill.

Adopting SWA to speed up a 4K-context model (attention wasn't the cost — see $L^* \approx 6d$), or expecting FlashAttention to shrink serving memory (it doesn't touch the cache). Diagnose with Figure 13.1 before prescribing.

GQA head-pooling without uptraining.

Mean-pooled KV heads produce a model that mostly works — the most dangerous kind of broken. The 5% uptrain is not optional.

Naive MLA implementation.

Computing explicit per-head K,V from the latent every step forfeits the entire trick; the query-absorption form is the algorithm, not an optimization of it. Likewise, applying standard RoPE to the latent path breaks the factorization — the decoupled key exists for a reason.

KV replication under tensor parallelism.

h_{kv} not dividing TP degree quietly multiplies your cache back up. Check the actual per-GPU cache allocation, not the formula.

Window smaller than the chat template.

A 1K window with a 3K system prompt means most layers literally cannot see the instructions by the end of a long generation. Size W against real prompt lengths, and keep global layers if instructions must stay visible.

Benchmarking prefill and calling it decode.

Prefill is compute-bound and loves FLOP reductions; decode is bandwidth-bound and loves cache reductions. A variant can win one and lose the other; report both.

13.8 Decision framework

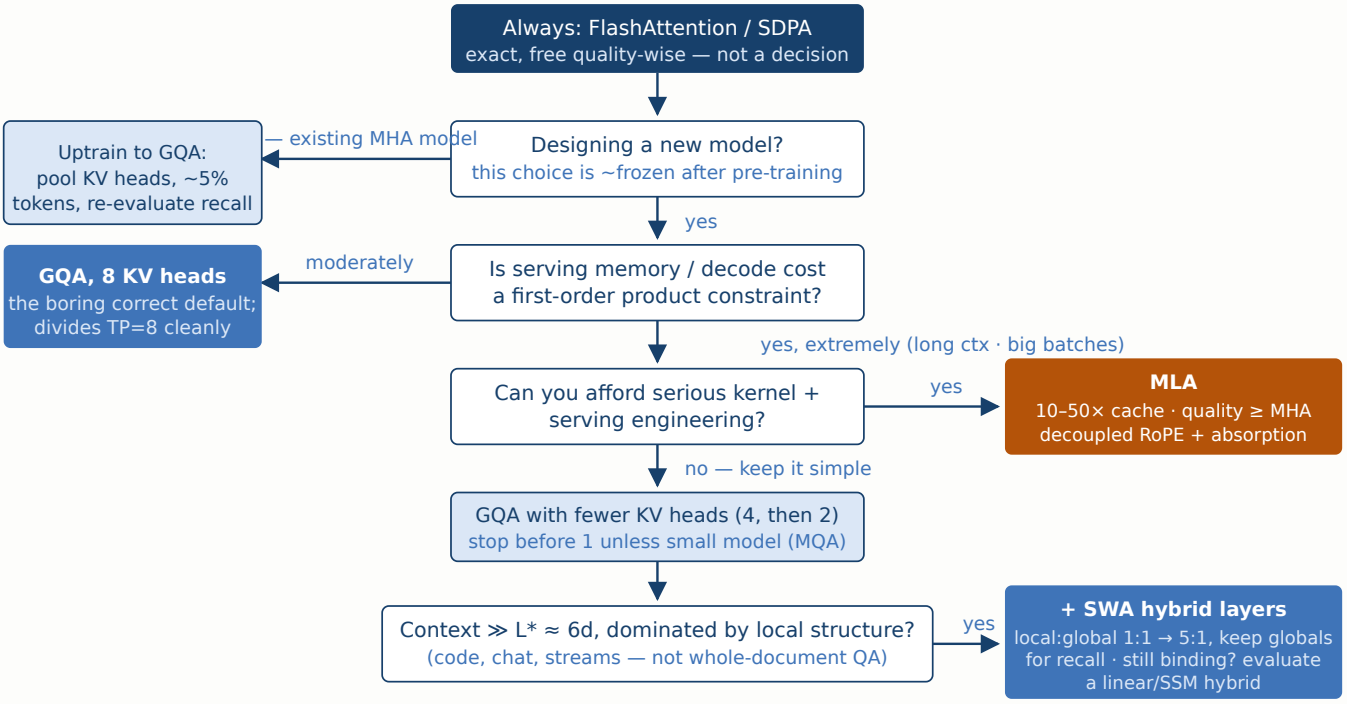

Figure 13.6 — Choosing an attention mechanism. The vertical spine is the design-time path for a new model; branches peel off for retrofits and for escalating constraints. Note what is *not* here: quality reasons to prefer plain MHA. In 2026 those are essentially gone above toy scale — the choice is between flavors of cheaper.

Cost-benefit, bluntly. FlashAttention: zero risk, large win, one line — there is no decision. GQA at design time: zero marginal cost (you were choosing head counts anyway), 4–8× decode benefit — also barely a decision. GQA by uptraining: ~5% of pre-training compute plus an evaluation cycle — cheap insurance for a serving-bound MHA model. SWA hybrids: cheap to implement, but they change what the model *can do* at long range, so their real cost is evaluation burden — pay it. MLA: weeks of engineering and ongoing serving-stack complexity, repaid only when cache is the binding constraint at scale — which for a big-context product it very much is. Linear hybrids: a research bet with a real payoff curve, made when even the above leaves you bandwidth-bound.

How this scales. The cache formula is $2Nh_{kv}d_h$ per token — it grows with depth and width, while HBM per GPU grows slowly, so the same product requirements bind harder at 70B than at 7B: a config that tolerates MHA at 7B may *require* GQA at 70B and make MLA attractive at DeepSeek scale (128 heads is why their 57× exists). Meanwhile the quadratic term binds harder as context, not model, grows — so the window/sparse family's payoff scales with your context ambitions, and the crossover $L^* \approx 6d$ moving up with d means bigger models actually tolerate longer contexts before the FLOP bill turns quadratic-dominated. Quick wins first: SDPA/Flash today, GQA-8 in the next config you freeze. Full build only when the bill says so.

13.9 Summary

Attention presents three bills, and confusing them wastes money: quadratic FLOPs that only start to matter past $L^* \approx 6d$, an $L \times L$ training-memory artifact, and — the one that dominates serving — decode's obligation to stream the whole KV cache per generated token at ~1 FLOP per byte. FlashAttention erases the memory artifact exactly, by tiling through SRAM with an online softmax, and is simply always on. The cache shrinks by consolidating KV heads: MQA (÷h, with a quality tax), GQA (÷h/g, the industry default at 8 groups, reachable from MHA checkpoints by a 5% uptrain), or DeepSeek's MLA (cache a 576-number latent per token, attend on it directly via query absorption — 57× with quality intact, at maximum engineering cost). Sliding windows cap both compute and cache but trade away direct long-range sight, so production systems interleave local and global layers; learned sparsity and linear/SSM hybrids continue the same trade further out. Choose by which bill binds: Flash always, GQA-8 by default, MLA when serving memory is the product, windows when context is long and local, hybrids when all else still isn't enough — and whatever you choose, evaluate long-range recall, because that is the currency every one of these techniques is quietly spending.

References

1. Pope, R., Douglas, S., Chowdhery, A., et al. (2022). Efficiently Scaling Transformer Inference. arXiv:2211.05102. The memory-bandwidth analysis of decoding: KV-cache traffic, not FLOPs, governs generation throughput at scale.
2. Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS*. arXiv:2205.14135. Tiling + online softmax; 3× GPT-2 training speedup, 15% over the MLPerf 1.1 BERT record, memory O(L).
3. Dao, T. (2023). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691. ~2× over FlashAttention-1, reaching ~70% of peak on A100.
4. Shazeer, N. (2019). Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150. Multi-query attention.
5. Chowdhery, A., et al. (2022). PaLM: Scaling Language Modeling with Pathways. arXiv:2204.02311. 540B model with multi-query attention for decode efficiency.
6. Ainslie, J., Lee-Thorp, J., de Jong, M., et al. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *EMNLP*. arXiv:2305.13245. Grouped-query attention; MHA→GQA conversion by mean-pooling KV heads plus ~5% uptraining.
7. Meta AI (2024). The Llama 3 Herd of Models. arXiv:2407.21783. GQA with 8 KV heads across the family (32:8 at 8B, 64:8 at 70B).
8. Jiang, A. Q., et al. (2023). Mistral 7B. arXiv:2310.06825. GQA (32:8) plus sliding-window attention with W = 4096 and a rolling-buffer cache.
9. DeepSeek-AI (2024). DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434. Introduces multi-head latent attention: KV compressed to a 512-dim latent plus a 64-dim decoupled RoPE key; ~93% KV-cache reduction with quality above MHA.

10. DeepSeek-AI (2024). DeepSeek-V3 Technical Report. arXiv:2412.19437. MLA at 671B/37B-active scale; 61 layers, latent 512 + 64.
11. Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The Long-Document Transformer. arXiv:2004.05150. Sliding-window attention with task-motivated global tokens.
12. Gemma Team, Google (2024). Gemma 2: Improving Open Language Models at a Practical Size. arXiv:2408.00118. Alternating local (W = 4096) and global attention layers, 1:1.
13. Gemma Team, Google (2025). Gemma 3 Technical Report. arXiv:2503.19786. 5 local layers (W = 1024) per global layer, sharply reducing long-context KV cache.
14. Child, R., Gray, S., Radford, A., & Sutskever, I. (2019). Generating Long Sequences with Sparse Transformers. arXiv:1904.10509.
15. Yuan, J., et al., DeepSeek-AI (2025). Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. arXiv:2502.11089.
16. Katharopoulos, A., Vyas, A., Pappas, N., & Fleuret, F. (2020). Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. *ICML*. arXiv:2006.16236.
17. Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752.
18. Lieber, O., et al., AI21 Labs (2024). Jamba: A Hybrid Transformer-Mamba Language Model. arXiv:2403.19887. One attention layer per seven Mamba layers at 52B (12B active).
19. Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS*. arXiv:1706.03762.
20. Touvron, H., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288.
21. OLMo Team, Allen Institute for AI (2024). 2 OLMo 2 Furious. arXiv:2501.00656. The 7B model retains full multi-head attention.
22. Qwen Team (2024). Qwen2.5 Technical Report. arXiv:2412.15115. GQA throughout; 28 query : 4 KV heads at 7B.